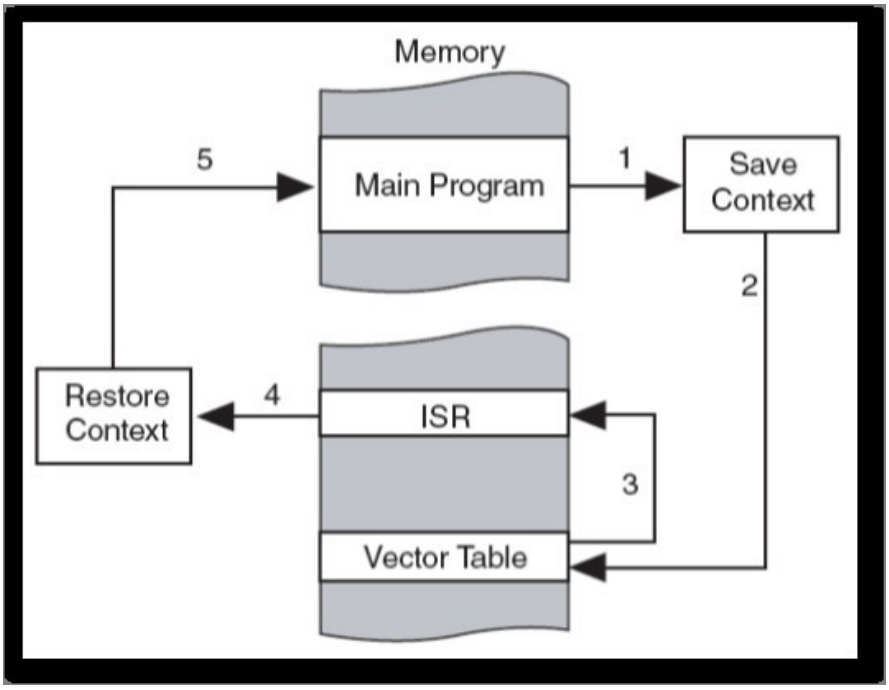
Title: Programming interrupts and using timers
Author: Craig Duffy 18/03/15, 15/01/18
Module: Mobile and Embedded Devices, Secure Embedded Systems
Awards: BSc CSI, Forensic Computing
Prerequisites: Basic computer architecture and some C

In our previous worksheet we saw how we could get input from a switch or button and send output to an LED. For such simple programs we could just loop round and continually test the state of the buttons to see what is going on and then do the required output. This method of operating is known as dedicated polling, which means going around asking the devices if the have in input or, if output devices, if they are ready to accept new data for output. This method of operating has the benefit of being simple to implement but is very limited and inefficient. If there are lots of devices then the polling loop could become very big and this might mean that devices won't get serviced when necessary. This can mean either delayed or lost input. For example if a keyboard was an input device at the end of a long polling loop then either it would appear unresponsive and then suddenly spew out data or data might get lost, leading to all sorts of problems and user frustrations. If it is crucially important that the data doesn't get lost or delayed then this system would be a real problem. An alternative method is busy-wait polling – in the keyboard example the system stalls waiting for data from the keyboard in order not to miss it. This then has the problem that other devices won't get serviced. As more and more devices were attached to computers this problem got bigger and bigger. Alternatively, the devices can be polled on a timed schedule, called periodic polling.

Perhaps a better solution is to get the devices to 'tell' the processor when they are ready – either to receive or transmit data. This allows the processor to get on with other tasks and not to waste time polling However this means that the processor must be able to support interrupts and have mechanisms for dealing sensibly with them. For example what happens to the interrupted process? What happens when the interrupt gets interrupted? What happens when several interrupts happen simultaneously? The provision of interrupts requires extra hardware, interrupt lines into the processor and priority levels to decide which interrupt to respond to. Earlier systems had limited numbers of interrupts and programming them required some skill. For example all Interrupt Services Routines (ISRs) would have to be coded in assembler and certain features and constructs were not allowed.

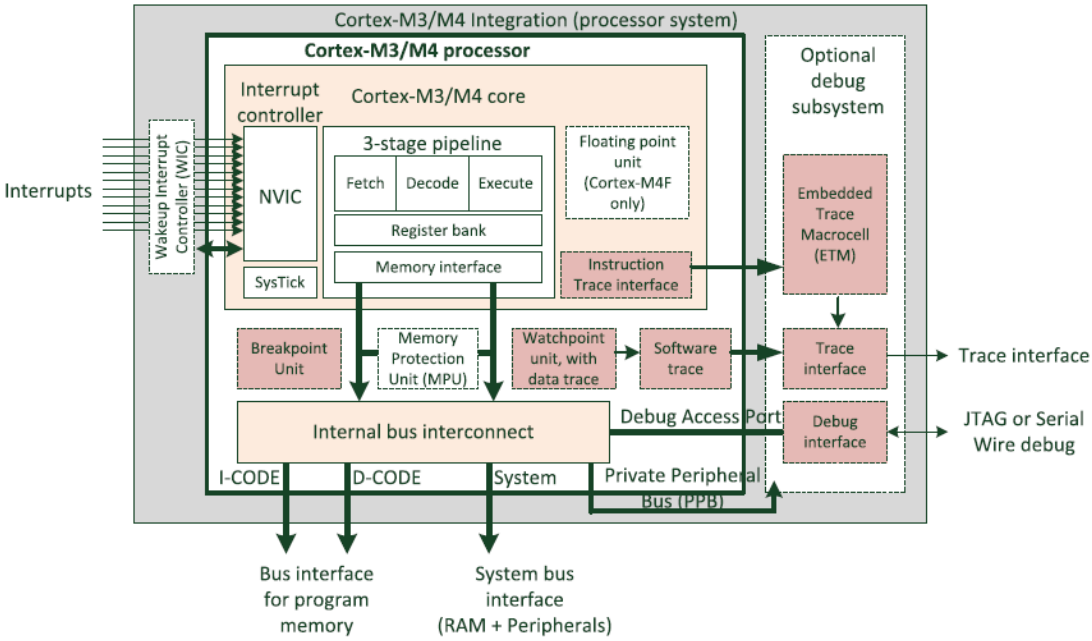


The system also has to decide what to do about the task that was interrupted. At some point the task will need to be resumed. In order to allow the resumption of interrupted tasks a certain amount of data needs to be saved, this data, known as the volatile environment or context, is the minimum amount of data required to restart the task. This would normally include CPU registers such as the program counter (PC), stack pointer (SP), link register (LR) and frame counter (FP). Other registers and data might need saving too, for example if a task is manipulating a screen the X and Y co-ordinates will need saving. This data will differ from system to system and will be affected by the language the code is written in and the operating system the code is being executed upon.

The provision of interrupts allows us to run more complex software such as operating systems and to support more complex hardware sub-systems such as direct memory access (DMA) or devices such as network interface or video cards. As such systems and hardware gets increasingly complex the interrupt support systems have had to evolve to support this. This means that systems have to cope with large number of interrupts, be able to allocate priorities to interrupts, block some interrupts and make sure that the correct code is matched with its interrupt.

Interrupts on the CORTEX M3

The Cortex M3 takes a quite different approach to interrupt handling from earlier ARM cores, now behaving much more like Intel processors. The interrupts are all handled by the **NVIC** (Nested Vectored Interrupt Controller). This handles all interrupts and allows code developed on one core to be reasonably portable to another. It also allows all of the ISRs, (Interrupt Service Routines), to be coded in C, saving the programmer from having to use assembler code, which to date was the normal method of coding ISRs.



In our previous worksheets we have already created and used ISRs as there are some that the system needs to get started. These were set up in the file **startup_stm32f10x.c**. In the declaration **void (* const g_pfnVectors[])(void)** the list of vectors is set up and initialised. The majority of the vectors are set to the default handler (**default_handler**) which contains a do nothing loop. However there are a number of entries that need to be filled with functional code.

1 Reset	-3	Reset_Handler
2 NMI	-2	NMI_Handler
3 Hard Fault	-1	HardFault_Handler
4 MemManage Fault	Prg	MemManage_Handler
5 Bus Fault	Prg	BusFault_Handler
6 Usage Fault	Prg	UsageFault_Handler
11 SVC	Prg	SVC_Handler
12 Debug Monitor	Prg	DebugMon_Handler
14 PendSV	Prg	PendSV_Handler
15 SYSTICK	Prg	SysTick_Handler
16 Interrupt #0	Prg	(device-specific)
17 Interrupt #1 - #239	Prg	(device-specific)

ARM Cortex interrupts

The first 15 are all fixed system interrupts - the lower the number the higher the priority. The ones without numbers (Prg), have programmable priorities these can be set, to allow interrupting devices to have higher or lower priority dependant upon their function. The vector which we have already initialised is the **Reset_Handler**, which is the code which sets up the program's execution environment and hands over to **main**.

In order to use interrupts we need to include some header and C files in our source and Makefiles. Most of the NVIC code is in **misc.c/h** but we will also need some code from the files **stm32f10x_gpio.c/h** and **stm32f10x_exti.c/h**. This is because we will require some of the pins in the GPIO area to be used for interrupts and the code in **stm32f10x_exti** is for external interrupts which we will use when we interface to buttons or serial ports. The set of functions provided for the NVIC are given in the table below. The main data structures for interrupts are **EXTI_InitTypeDef EXTI_InitStructure** and **NVIC_InitTypeDef NVIC_InitStructure**. These are described in the respective header files.

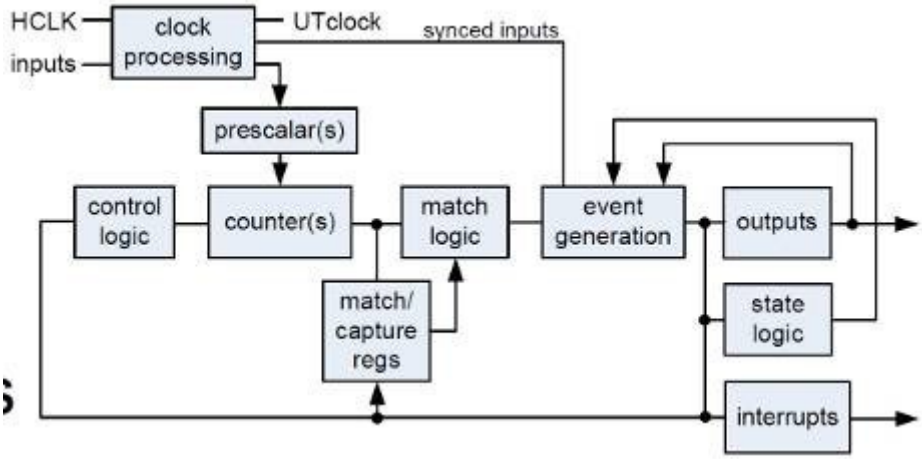
CMSIS function(1)	Description
void NVIC_EnableIRQ(IRQn_Type IRQn)	Enables an interrupt or exception.
void NVIC_DisableIRQ(IRQn_Type IRQn)	Disables an interrupt or exception.
void NVIC_SetPendingIRQ(IRQn_Type IRQn)	Sets the pending status of interrupt or exception to 1.
void NVIC_ClearPendingIRQ(IRQn_Type IRQn)	Clears the pending status of interrupt or exception to 0.
uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn)	Reads the pending status of interrupt or exception. This function returns non-zero value if the pending status is set to 1.
void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)	Sets the priority of an interrupt or exception with configurable priority level to 1.
uint32_t NVIC_GetPriority(IRQn_Type IRQn)	Reads the priority of an interrupt or exception with configurable priority level. This function return the current priority level.

1. The input parameter IRQn is the IRQ number,

NVIC access functions

Timers

Before we look at some interrupt code we need to briefly look at the timers that the Cortex provides. As we have already seen timers are crucial for the operation of computer systems. The Cortex has a wide range of timers available. We are going to configure a timer to generate our interrupts and flash an LED (what else!). Timers require a clock input, they have a pre-scaler, which is used to divided the crystal clock signal, they normally have a target value that they want to achieve and a series of actions to be undertaken on reaching the target.



The Cortex timers can either get their clock signal from an external source or use the main CPU clock, which can run between 2 and 72Mhz and this can be divided down by a 16 bit pre-scaler value (0..65535). The match logic can be count up, down or up/down, On reaching its target it can either reload and start again or stop. The timers can generate an event such as a signal or an interrupt, which is what we are going to do.

The code to configure the timer is given below.

```
void configure_timer(void)
{
  RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
  TIM_TimeBaseInitTypeDef timerInitStructure;

  timerInitStructure.TIM_Prescaler = 36000;
  timerInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
  timerInitStructure.TIM_Period = 10000;
  timerInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
  timerInitStructure.TIM_RepetitionCounter = 0;

  TIM_TimeBaseInit(TIM2, &timerInitStructure);
  TIM_Cmd(TIM2, ENABLE);
  TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);
}
```

This structure is used by the library code to initialise the timer hardware. We are going to use timer 2. The pre-scaler value is pretty clear, the period is the value that is counted up to and is reloaded on reaching the target. The variable TIM_CLD_DIV1 tells the system to use the main clock – the main clock can be divided by other values. TIM_ITConfig configures the timer to generate interrupts. Finally the timer is enabled.

We next need to set up the interrupt for timer 2:

```
void configure_timer2_interrupt()
{
    NVIC_InitTypeDef nvicStructure;
    nvicStructure.NVIC_IRQChannel = TIM2_IRQn;
    nvicStructure.NVIC_IRQChannelPreemptionPriority = 0;
    nvicStructure.NVIC_IRQChannelSubPriority = 1;
    nvicStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&nvicStructure);
}
```

We are setting up the priorities very simply, as we don't have any other processes to worry about, but we will see later with FreeRTOS, that priorities can be used in very subtle ways to support system functionality.

Finally we need an interrupt service routine – ISR.

```
void TIM2_IRQHandler()
{
  if (TIM_GetITStatus(TIM2, TIM_IT_Update) != RESET)
  {
    TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
    GPIO_WriteBit(GPIOC , GPIO_Pin_6 , (greenledval) ? Bit_SET : Bit_RESET);
    GPIO_WriteBit(GPIOC , GPIO_Pin_7 , (greenledval) ? Bit_SET : Bit_RESET);
    greenledval = 1 - greenledval;
  }
}
```

The function name is fixed by the convention set up by ST Micro, in their header file **stm32/worksheets/STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport /ST/STM32F10x/stm32f10x.h**. These names are reflected in the names used in the vector table in the file **startup_stm32f10x.c**. There the names of all the vectors are given but they have the linker directive 'weak':

```
void TIM1_TRG_COM_TIM17_IRQHandler (void) __attribute__ ((weak, alias ("default_handler")));
void TIM1_CC_IRQHandler (void) __attribute__ ((weak, alias ("default_handler")));
void TIM2_IRQHandler (void) __attribute__ ((weak, alias ("default_handler")));
void TIM3_IRQHandler (void) __attribute__ ((weak, alias ("default_handler")));
```

This setting means that the name can be overridden in a later declaration. So our declaration is taken in preference. You will notice that the other ISR are all set to the default handler.

Exercise 1

Build the program and execute it. The program is called TimerInt.c either change its name to main.c or change the Makefile to build it with that name.

Try changing the values so that the lights flash every second.

Button interrupts for LED

Now we are going to look at connecting an external event with an interrupt. We will try now to create an interrupt that is triggered by pressing one of the buttons. The code below does the initialisation of the button and sets up the interrupt. Notice how the GPIO event needs to be tied to the interrupt - **GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource0):**

```
GPIO_InitTypeDef GPIO_InitStructure;
EXTI_InitTypeDef EXTI_InitStructure;
NVIC_InitTypeDef NVIC_InitStructure;

/* Initialise the GPIOA for button WKUP */
GPIO_InitStructure.GPIO_Pin =  GPIO_Pin_0;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
GPIO_Init( GPIOA, &GPIO_InitStructure );

/* Connect Button EXTI Line to Button GPIO Pin */
GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource0);

/*Configure Button EXTI line */
EXTI_InitStructure.EXTI_Line = EXTI_Line0;
EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising;
EXTI_InitStructure.EXTI_LineCmd = ENABLE;
EXTI_Init(&EXTI_InitStructure);

/* Configure interrupts for button */
NVIC_InitStructure.NVIC_IRQChannel = EXTI0_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init( &NVIC_InitStructure );
```

We also need an ISR for the button:

```
void  EXTI0_IRQHandler()
{
  if ( EXTI_GetITStatus(EXTI_Line0) == SET)
  {
    EXTI_ClearITPendingBit( EXTI_Line0 );

    GPIO_WriteBit(GPIOC , GPIO_Pin_7 , (yellowledval) ? Bit_SET : Bit_RESET);
    yellowledval = 1 - yellowledval;
  }
}
```

Obviously we will need to start_up the bus clock for both GPIOA and GPIOC, as we are using the button on GPIOA and the LED on GPIOC.

Exercise 2

Build the code – the program is called buttona.c, either rename or edit the Makefile as in the previous exercise.

Initialise the button on GPIOC and tie that to an ISR which flashes the Green button.

UART interrupts

Dealing with an interrupt when it is a single button press to switch on a single LED is pretty easy. Dealing with more complex input can be much more tricky. We are going to look at asynchronous interrupt driven IO. This means that the routines to do the IO are driven by interrupts, and they will operate when the devices are ready. Unlike the polled versions we have seen previously these are detached from the underlying processes that require the IO channel. We need to add interrupt capability to the IO structure of Worksheet4!

Before we get to that level of complexity first let's try to get some interrupt driven UART code written. To start using the interrupt driven UART we will need some of the library data structures:

```
USART_InitTypeDef USART_InitStructure;
GPIO_InitTypeDef GPIO_InitStructure;
NVIC_InitTypeDef NVIC_InitStructure;
EXTI_InitTypeDef EXTI_InitStructure;
```

We also need to set up the UART to generate interrupts as well as to deal with input and output.

```
/* Enable GPIO clock */
RCC_APB2PeriphClockCmd( RCC_APB2Periph_GPIOD | RCC_APB2Periph_AFIO, ENABLE );

/* Enable USART2 clock */
RCC_APB1PeriphClockCmd( RCC_APB1Periph_USART2, ENABLE );

/* Remap USART, as USART2 uses alternate pins */
GPIO_PinRemapConfig( GPIO_Remap_USART2,ENABLE );

/* Configure USART2 Tx pin */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5; /*changed 8 to 5 for USART2 craig */
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
GPIO_Init( GPIOD, &GPIO_InitStructure );

/* Configure USART2 Rx pin */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6; /* changed 9 to 6 for USART2 craig */
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
GPIO_Init( GPIOD, &GPIO_InitStructure );

GPIO_EXTILineConfig(GPIO_PortSourceGPIOD, GPIO_PinSource6);

/* Enable USART */
USART_Cmd( USART2, ENABLE );

/* Configure USART 8N1 */
USART_InitStructure.USART_BaudRate = mainCOM_PORT_BAUD_RATE;
USART_InitStructure.USART_WordLength = USART_WordLength_8b;
USART_InitStructure.USART_StopBits = USART_StopBits_1;
USART_InitStructure.USART_Parity = USART_Parity_No;
USART_InitStructure.USART_HardwareFlowControl =
USART_HardwareFlowControl_None;
USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;
USART_Init( USART2, &USART_InitStructure );

USART_ITConfig(USART2, USART_IT_RXNE, ENABLE);
```

We can see that most of the settings are the same but with the added complexity of setting up the interrupts. (Note: we have enabled receive (RX) interrupts only!) We need to set up the NVIC too.

```
/* Configure interrupts for usart  */
NVIC_InitStructure.NVIC_IRQChannel = USART2_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init( &NVIC_InitStructure );

NVIC_EnableIRQ(USART2_IRQn);
```

All of this code needs putting in a ComPortInit function to be called early on in main's execution. However we still need an interrupt service routine for the port.

```
void USART2_IRQHandler()
{
  if ( (USART_GetITStatus(USART2, USART_IT_RXNE) != RESET ) )
  {
    data = (int) USART_ReceiveData(USART2) & 0xff;
    USART_ClearITPendingBit(USART2, USART_IT_RXNE);
  }
}
```

If we put all this together then we have a small problem, how will the main program know when data has been received and how does the main program find that data? Therefore we will need some way of communicating between the ISR and the main program. The simplest method is use a global variable to indicate when an interrupt has been fired. Alternatively the operating system task signalling facilities could be used, but here we are starting with the simplest option. The changes are given below.

```
.............…
int gotit=0, rxdata=0;
..............…
void USART2_IRQHandler()
{
  if ( (USART_GetITStatus(USART2, USART_IT_RXNE) != RESET ) )
  {
    rxdata = (int) USART_ReceiveData(USART2) & 0xff;
    gotit=1;
    USART_ClearITPendingBit(USART2, USART_IT_RXNE);
  }
}
................…
```

Finally we need to change the main function.

```
int main(void)
{
  COMPortInit();
  while (1)
  {
    if (gotit == 1 )
    {
      outbyte(rxdata);
      gotit=0;
    }
  }
}
```

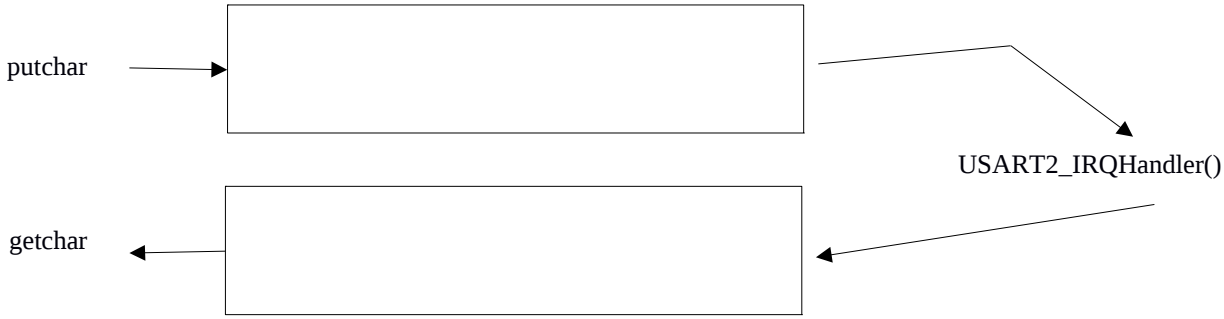
In this way we have simply substituted polling a software flag (gotit) for polling a hardware status bit (USRT_IT_RXNE), which may not appear much of a gain.

Exercise.

Build the interrupt driven input program and test it. (Don't forget to allow for any new library routines that are used – check your includes and the Makefile.) Make the output interrupt driven. This will require the transmit lines setting up to generate interrupts. Also the interrupt routine will have to check both the RX and TX flags to decide what to do.

Producer/consumer relationship

Having a 1 byte buffer is of limited use, to make IO more efficient a large buffer is useful. This means that a program doesn't have to wait for the UART to become ready to write data, it just puts it in a buffer, the larger the buffer the more data that can be queued up for when the UART is ready. Likewise the receive routine just looks into a buffer fed by the UART's interrupt driven receive routine. This separation means that the processes dealing with the input are working at a distance from the actual IO routines.



However we then get into the a situation where some new problems can occur. The data buffers are termed <u>critical resources</u> because they are accessed asynchronously by separate tasks, which means data corruption could easily occur if the interrupt routine activates in the middle of a putchar/getchar operation. Also, what happens if the output buffer is full because the UART is not transmitting? The process then has a choice, it can either poll and await for a buffer space or record an error. It will also need to signal to the IRQHandler that there is data ready to transmit otherwise that routine will be continually firing off interrupts even when its output buffer is empty. The problem for getchar is the same in reverse. The interrupt routine will be signalled by the arrival of data into the UART however it won't have control over the buffer, so it has to decide what to do if that is full – and it is either poll or drop.

Credit exercise.

Implement full interrupt driven IO with a small buffer of about 20 characters. The putchar and getchar routines should either put or remove data into the buffer. The should be able to deal with a full buffer and an empty one. The ISR should be able to deal with no characters to send.

Doing Pulse Width Modulation (PWM)

The timers can also be used to create PWM signals which are very important in DC motor control. PWM provides powered control of analogue devices by digital computers through quickly chopping the current on and off. We are going to use the PWM capabilities of the STM32 timers to slowly fade and brighten the LEDs on the Olimex board.

In order to do this we need to set up one of the timers into PWM mode.

```
void configure_timer(void)
{
  TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStruct;
  TIM_OCInitTypeDef TIM_OCInitStruct;

  RCC_APB1PeriphClockCmd( RCC_APB1Periph_TIM3, ENABLE);
  RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC |RCC_APB2Periph_AFIO, ENABLE);

  /* Configure PC6 in output pushpull mode */
  GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;   // Alt Function - Push Pull
  GPIO_Init(GPIOC, &GPIO_InitStructure);

  // Map TIM3_CH1 to GPIOC_Pin6
  GPIO_PinRemapConfig( GPIO_FullRemap_TIM3, ENABLE );

  // Let PWM frequency equal 100Hz.
  // Let period equal 1000. Therefore, timer runs from zero to 1000.
  // Gives 0.1Hz resolution.
  // Solving for prescaler gives 240.
  TIM_TimeBaseStructInit( &TIM_TimeBaseInitStruct );
  TIM_TimeBaseInitStruct.TIM_ClockDivision = TIM_CKD_DIV4;
  TIM_TimeBaseInitStruct.TIM_Period = 1000 - 1;            // 0..999
  TIM_TimeBaseInitStruct.TIM_Prescaler = 240 - 1;          // Div 240
  TIM_TimeBaseInit( TIM3, &TIM_TimeBaseInitStruct );

  TIM_OCStructInit( &TIM_OCInitStruct );
  TIM_OCInitStruct.TIM_OutputState = TIM_OutputState_Enable;
  TIM_OCInitStruct.TIM_OCMode = TIM_OCMode_PWM1;
  // Initial duty cycle equals 0%. Value can range from zero to 999.
  TIM_OCInitStruct.TIM_Pulse = 0; // 0 .. 1000 (0=Always Off, 999=Always On)
  TIM_OC1Init( TIM3, &TIM_OCInitStruct );

  TIM_Cmd( TIM3, ENABLE );
}
```

Timer and GPIO configuration

You will notice that we are going to use GPIO_Pin_6 (the green LED) and this is configured for output, also this pin is given an alternate function. This allows us to connect the pin to the output channel of the timer, thus giving us the modulated signal. Next we set up the timer into PWM mode and set the duty cycle of the PWM. This allows us to set the time that the LED is turned on as the intensity increases. Note that as well as setting up the reset and clock controller (RCC) for timer 3 and the GPIO we need to turn it on for the alternate function.

The code that runs this is fairly simple,

```
int main(void)
{
  int i;
  int n = 1;
  int brightness = 0;
  configure_timer();

  while (1)
  {
    if (brightness >= 1000)  n = -1; // if brightness maximum change direction

    if (brightness <= 0) n = 1;      // if brightness minimum change direction
    brightness += n;   // increment or decrement brightness refer by direction
    TIM3->CCR1 = brightness; // set brightness
    for(i=0;i<0x4000;i++) ;  // delay
  }
}
```

The main function

This code will obviously require some header files, an assert_failed() function as well as a Makefile linking in the correct library files. This is left as an exercise for the students.

A **credit exercise** is to get the PWM to fade and brighten the yellow and green LEDs in opposite directions!